$ git log --oneline main

De la CI *minimale* à la CI solide

01 **Ma première CI avec GitHub Actions** — TP1
Workflow Hello · ci.yml · échec volontaire · pull request · badge

02 **Une CI solide** — TP2
Lint · matrice · cache · artefact · protection de main · conflit · secret · pipelines à réparer

AUTEUR
Samir MAMA

DÉPÔT
damesys2008-ship-it/tasks-api

CI passing

01

$ git push origin tp1

Ma première CI *avec* GitHub Actions



OBJECTIF

Créer une CI à vérifier avec GitHub Actions, et ajouter un badge dans le README.

CAPTURES

7

DÉPÔT

damesys2008-ship-it/tasks-api

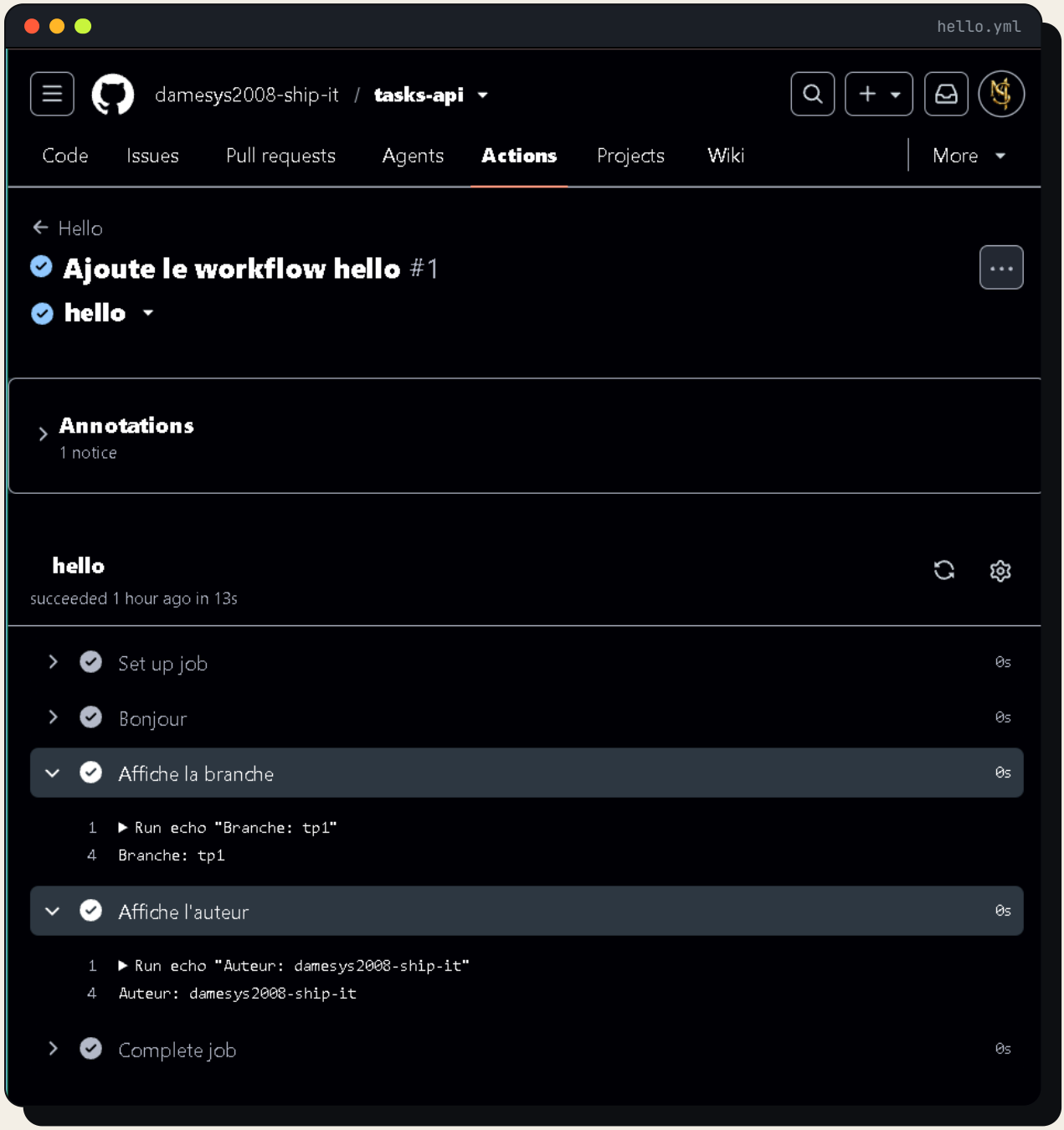
CI passing

TP 1-1

PASSED

Hello, world — version pipeline

Le run « *Hello* » réussi, avec les étapes `branche` et `auteur` dépliées.

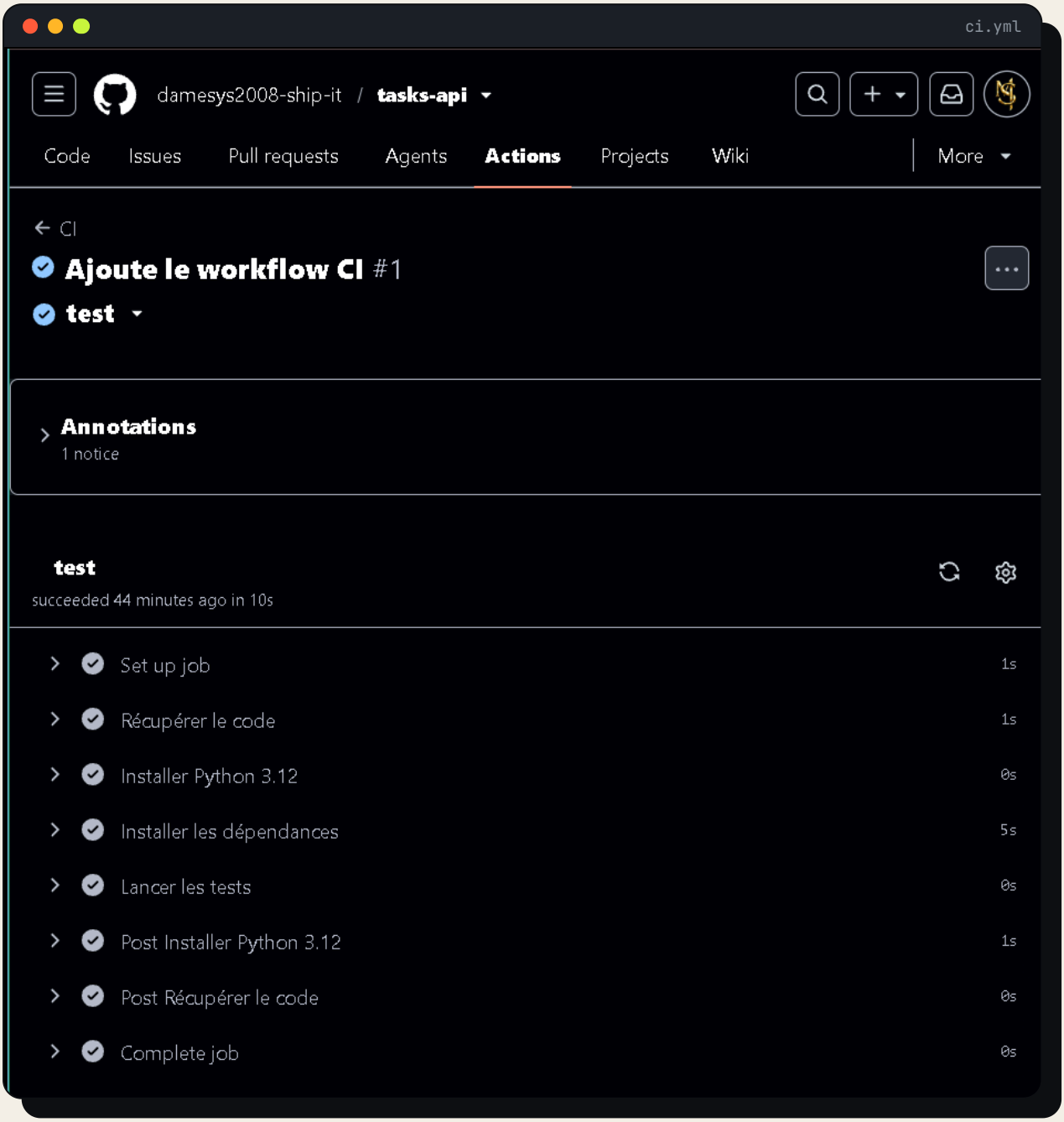


TP 1-2

PASSED

La vraie CI entre en scène

Le run `ci.yml` réussi, avec la liste complète des étapes.



TP 1-3

FAILED

On casse tout (exprès)

Provocation volontaire d'un échec de CI.

ci.yml

test

failed 47 minutes ago in 12s

Lancer les tests 0s

```
tests/test_api.py ...F.                                                  [ 50%]
tests/test_logic.py ..F..                                                [100%]

================================== FAILURES ===================================
_____________________________ test_add_and_list_tasks _____________________________

client = <FlaskClient <Flask 'app.main'>>

    def test_add_and_list_tasks(client):
        created = client.post("/tasks", json={"title": "  apprendre la ci "})
>       assert created.status_code == 201
E       assert 400 == 201
E        +  where 400 = <WrapperTestResponse streamed [400 BAD REQUEST]>.status_code

tests/test_api.py:29: AssertionError
____________________ test_is_valid_title_accepts_normal_title ____________________

    def test_is_valid_title_accepts_normal_title():
>       assert is_valid_title("Faire le TP1") is True
E       AssertionError: assert False is True
E        +  where False = is_valid_title('Faire le TP1')

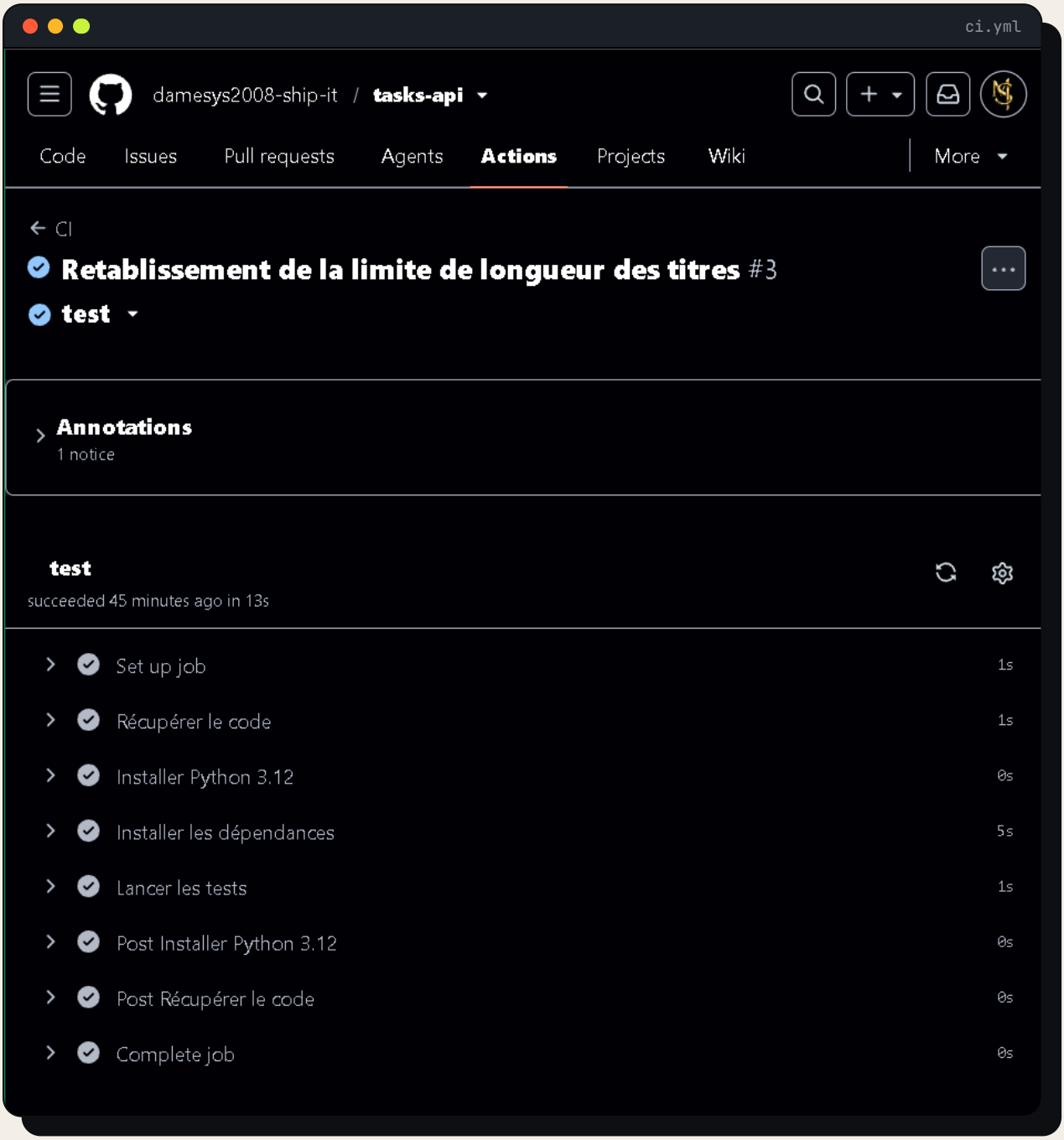
tests/test_logic.py:13: AssertionError
=========================== short test summary info ============================
FAILED tests/test_api.py::test_add_and_list_tasks - assert 400 == 201
 +  where 400 = <WrapperTestResponse streamed [400 BAD REQUEST]>.status_code
FAILED tests/test_logic.py::test_is_valid_title_accepts_normal_title - AssertionError: assert False is
True
 +  where False = is_valid_title('Faire le TP1')
======================== 2 failed, 8 passed in 0.20s ========================
Error: Process completed with exit code 1.
```

TP 1-4

PASSED

Retour au vert

Nouveau run après correction de l'échec.

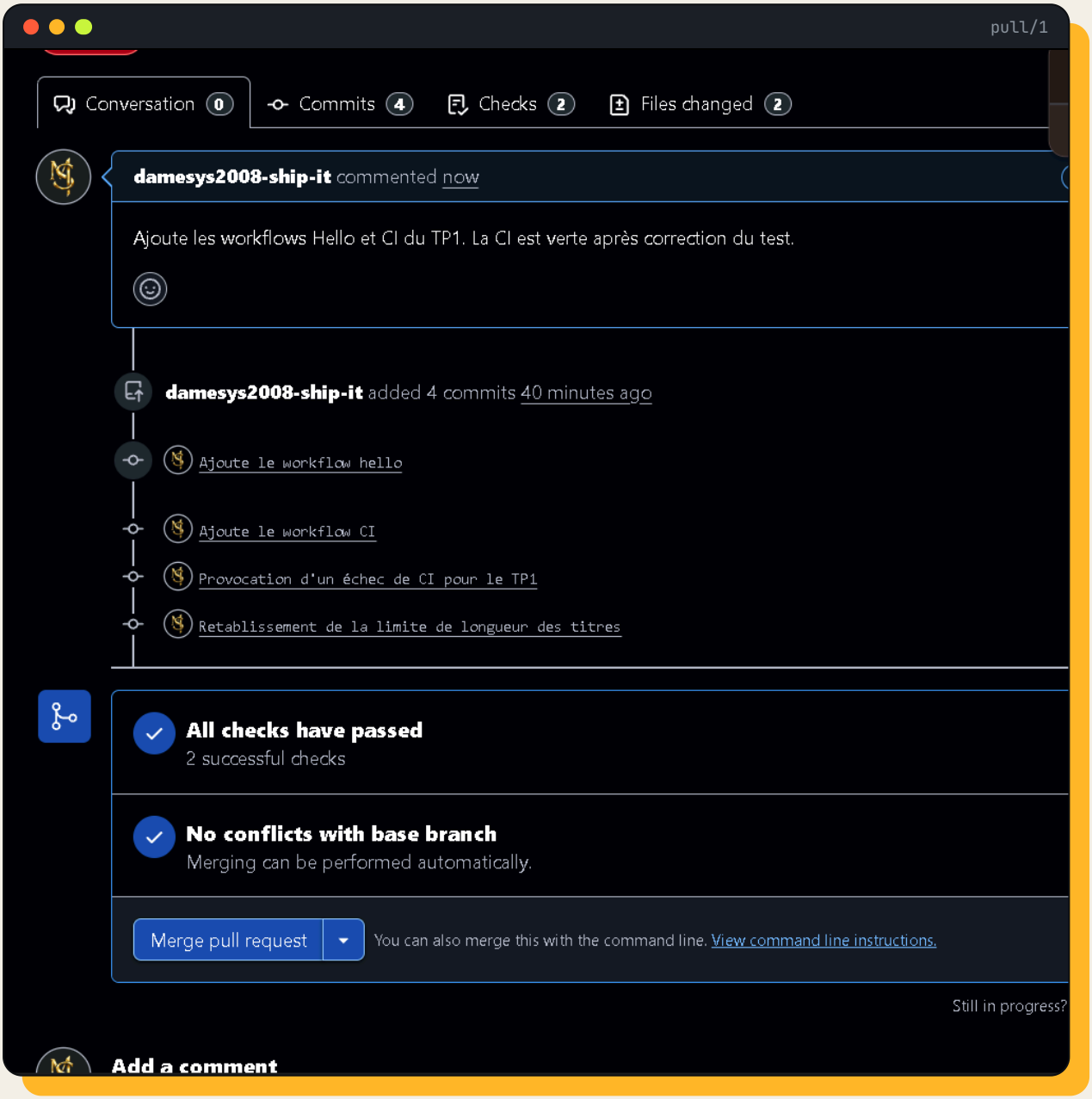


TP 1-5

CHECKS

Les checks font le videur

La pull request avec les vérifications avant la fusion.

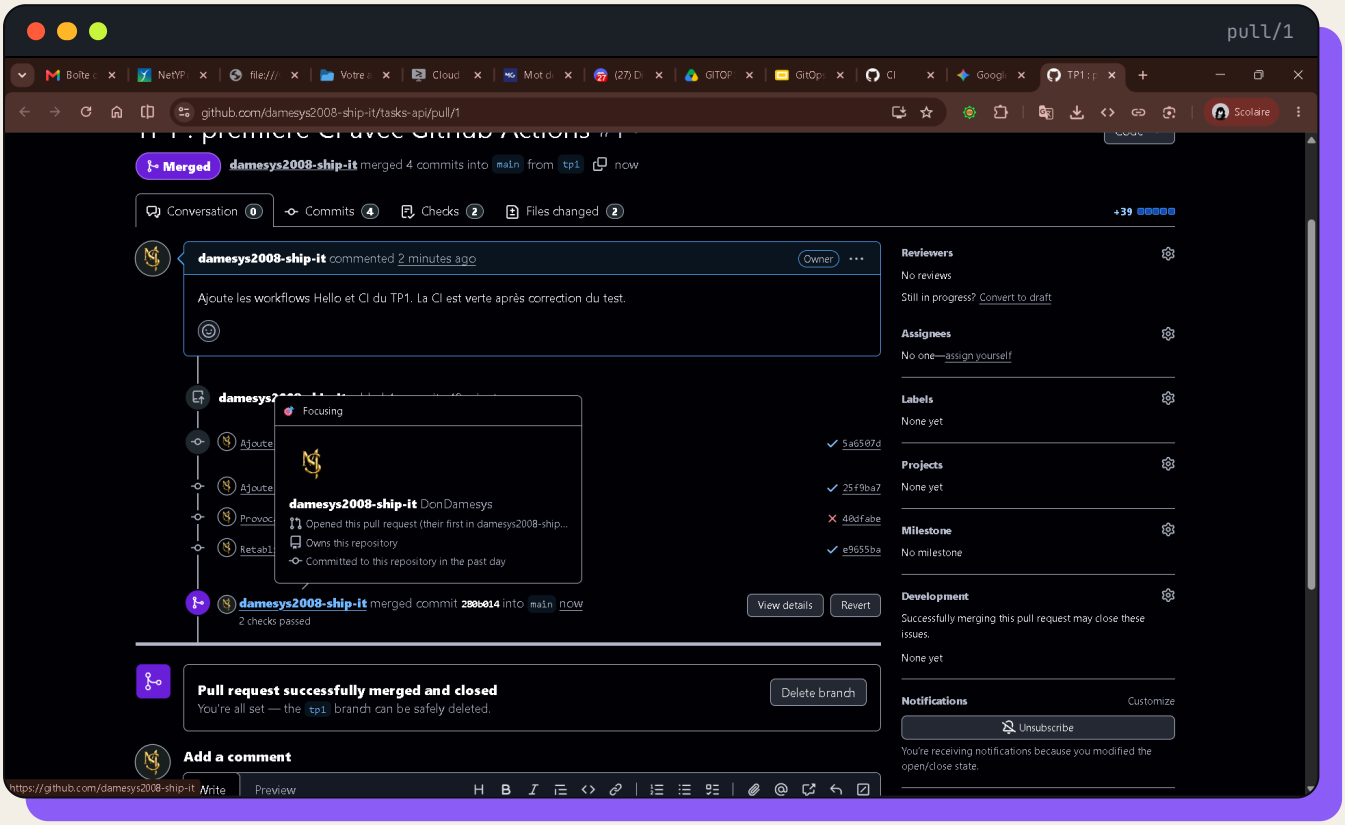


TP 1-6

MERGED

Fusion validée

La pull request après le merge dans `main`.



TP 1-7

PASSED

Le badge, la cerise

Le `README.md` sur GitHub avec le badge de CI affiché.



$ cat tp1/reponses.md

Réponses *aux questions — TP1*

01 Combien de temps a duré le run « Hello » ? Où voit-on les logs de chaque étape ?

Le run « Hello » a duré exactement **16 secondes**. Les logs, eux, se retrouvent en **ouvrant chaque étape** du job.

02 Quel test a échoué dans la partie 3, et à quelle ligne ? Qu'est-ce qui a permis de le trouver rapidement ?

Deux tests ont échoué : `test_add_and_list_tasks` et `test_is_valid_title_accepts_normal_title`. Le rapport de **pytest** affichait le nom des tests, les fichiers et les lignes, ce qui a permis de les retrouver rapidement.

03 Pourquoi attend-on que la CI soit verte avant de fusionner une pull request ?

Une CI verte avant la fusion confirme que les tests passent **sur une machine propre** et évite d'intégrer une modification qui casse le projet.

TP1 — all checks have passed · `exit 0`

02

$ git switch -c tp2

Une CI *vraiment* solide



OBJECTIF

Transformer la CI minimale du jour 1 en CI de qualité : analyse du code, couverture, plusieurs versions de Python, cache, protection de la branche `main`, travail à deux sur un même dépôt et gestion d'un secret.

CAPTURES
17

DÉPÔT
damesys2008-ship-it/tasks-api

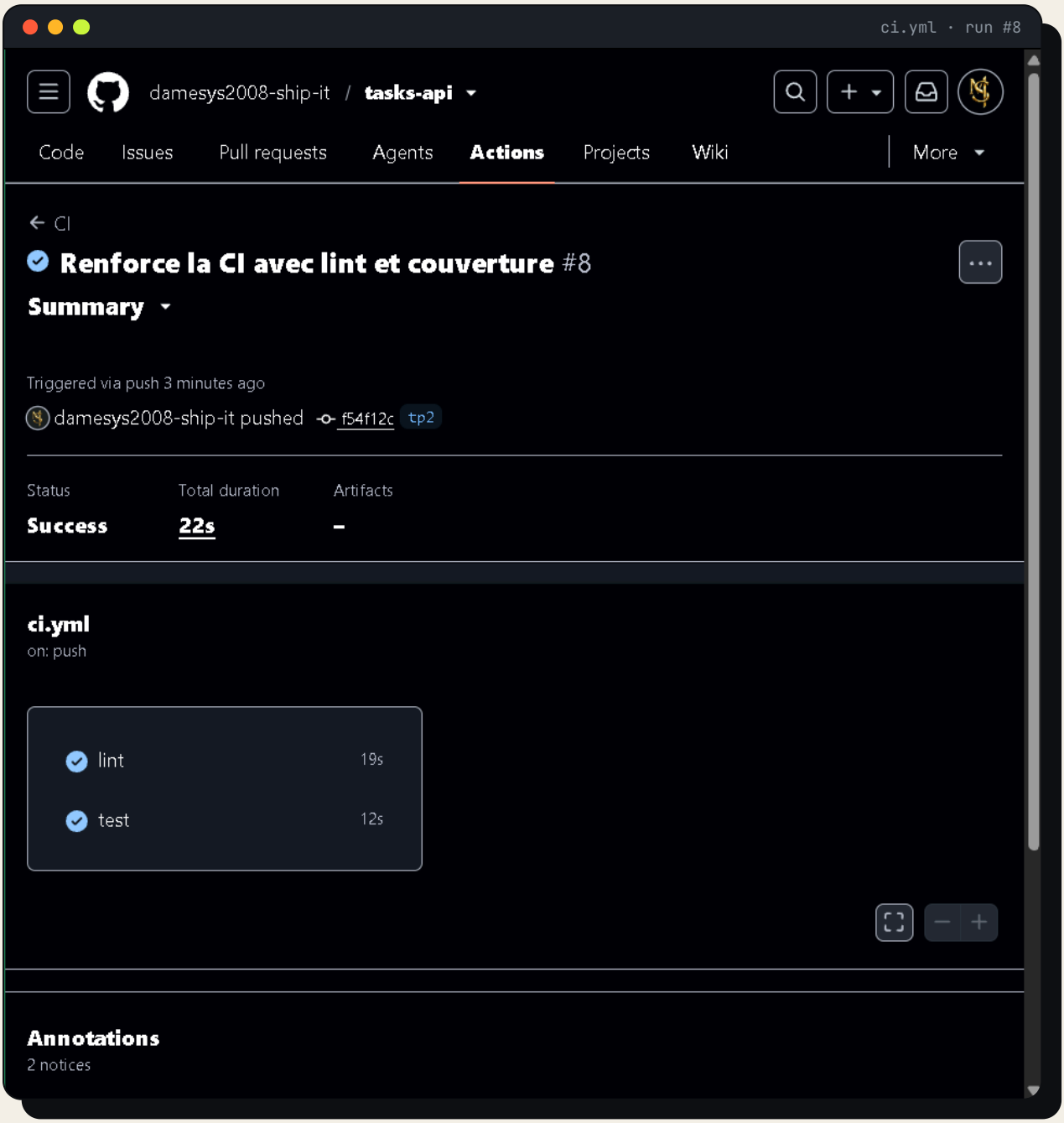
CI passing

TP 2-1

PASSED

Deux jobs valent mieux qu'un

Un run vert montrant les deux jobs `lint` et `test`.

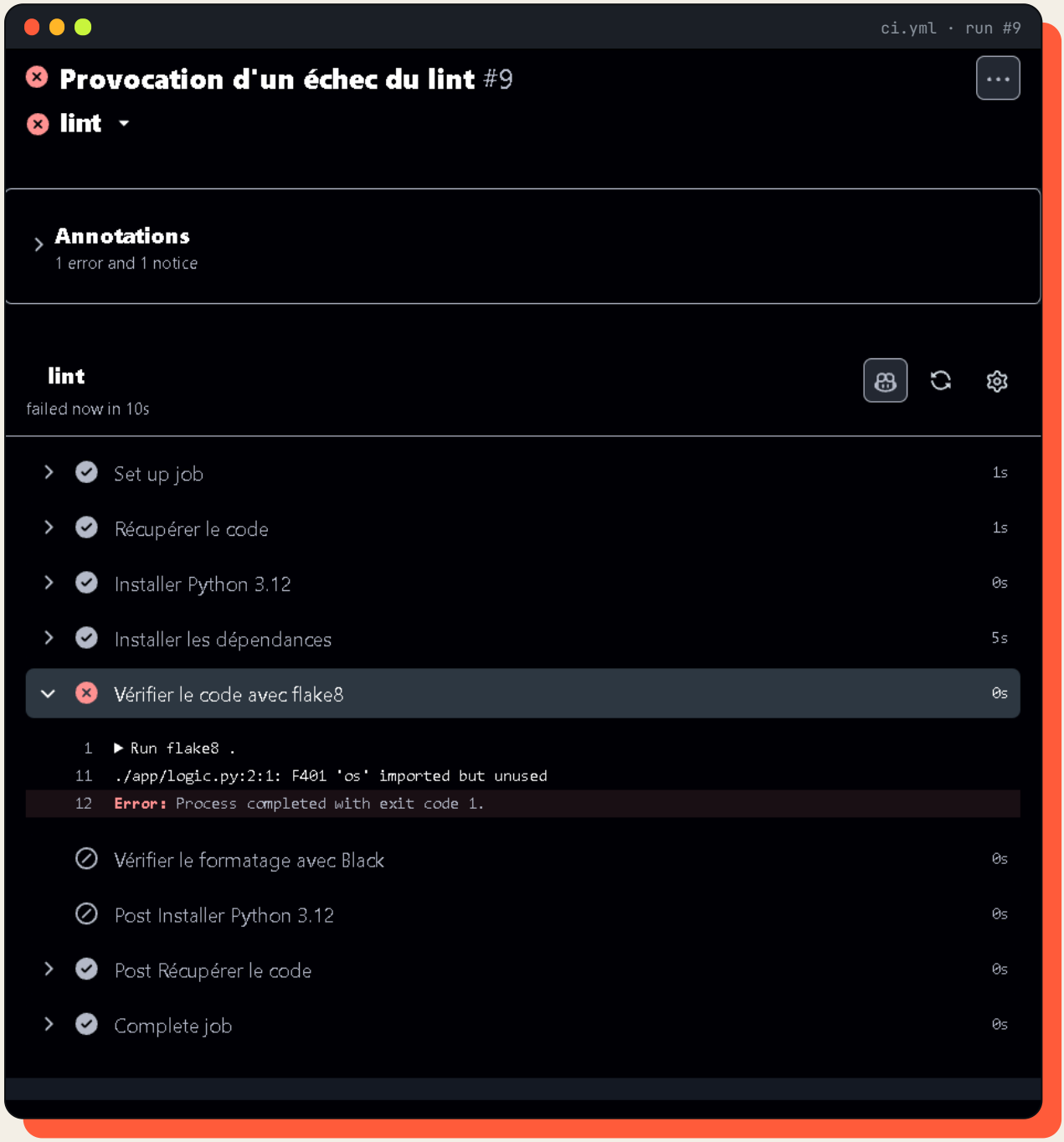


TP 2-2

FAILED

flake8 a l'œil

Le job `lint` rouge, avec le message de flake8 lisible : `F401 'os' imported but unused`.

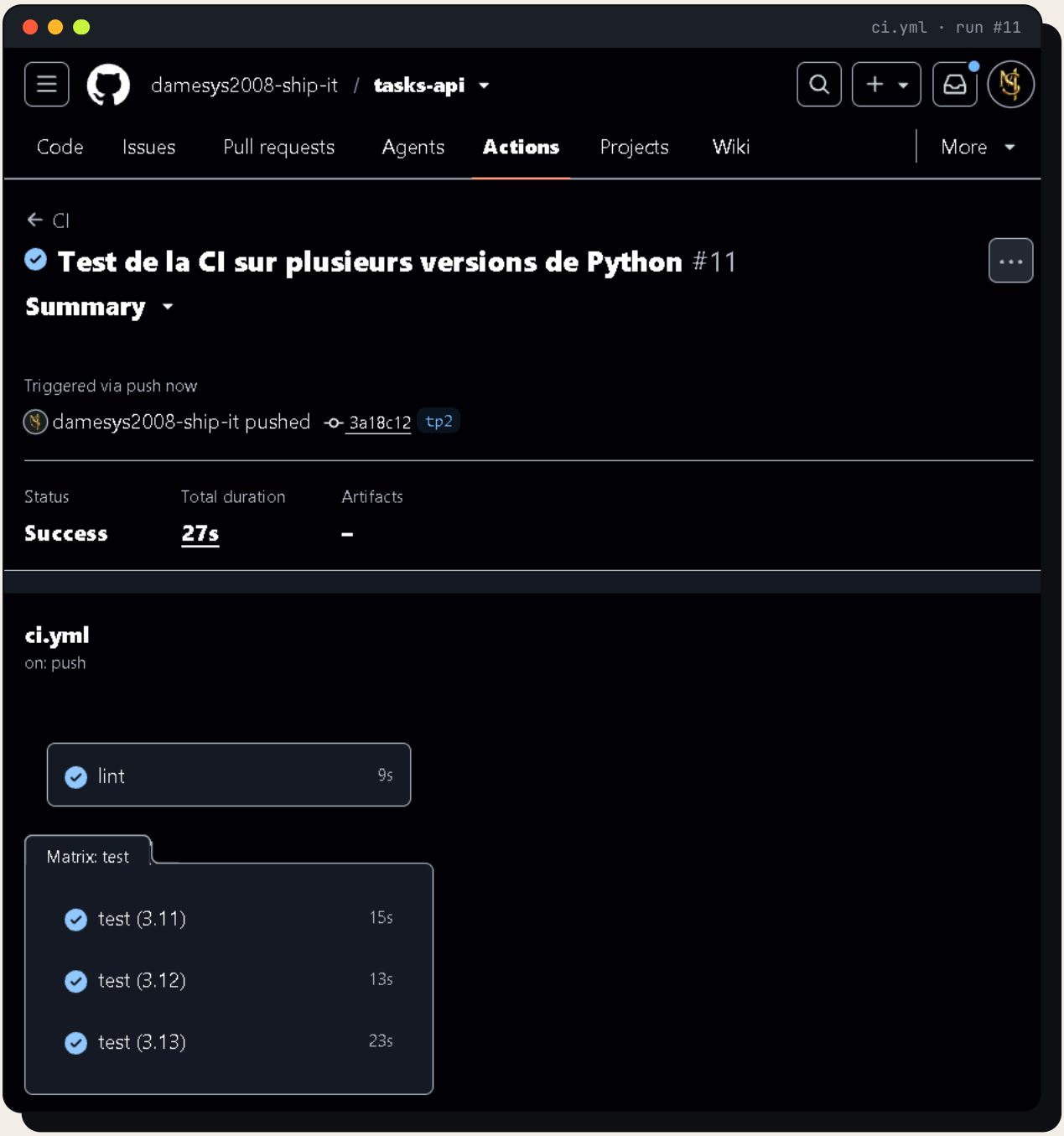


TP 2-3

MATRIX ×3

Trois Pythons, zéro jaloux

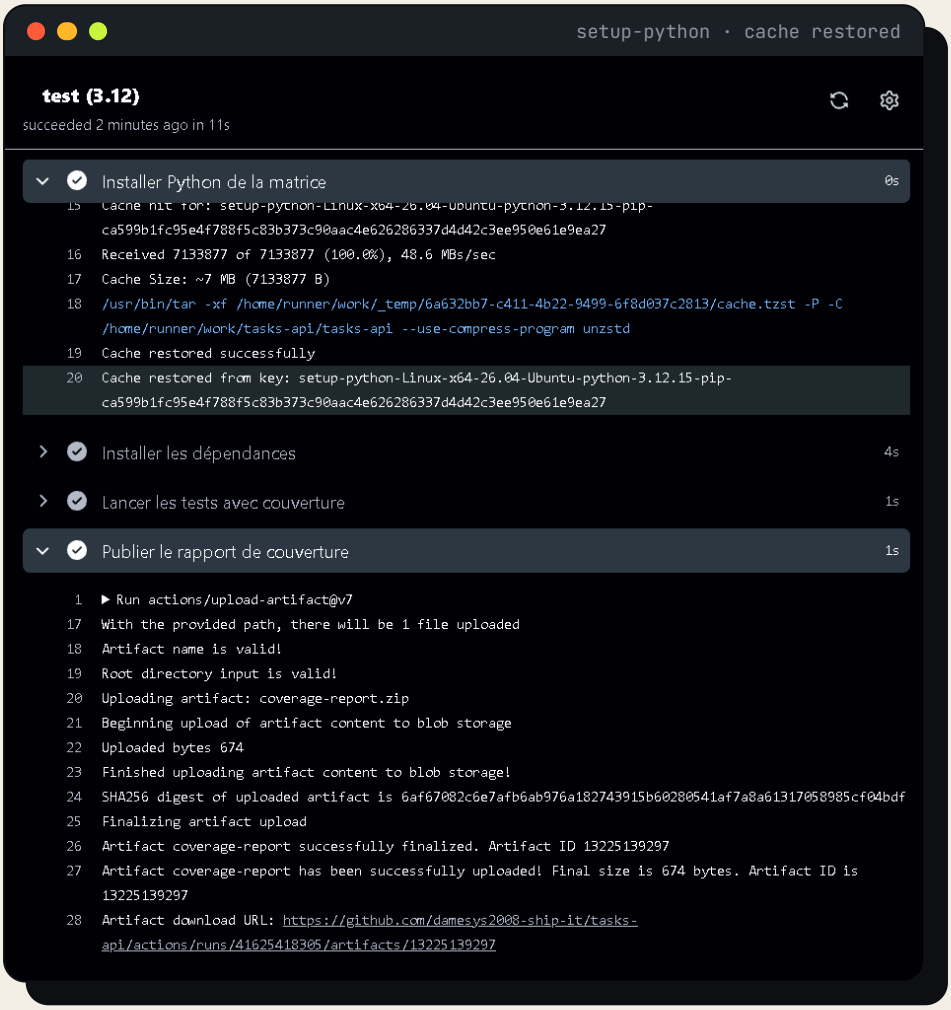
La matrice : `test (3.11)`, `test (3.12)` et `test (3.13)`.



CACHE + ARTEFACT

TP 2-4 Cache restauré, rapport emballé

L'étape setup-python du 2e run avec la mention du cache restauré, et la section « Artifacts » de la page du run.

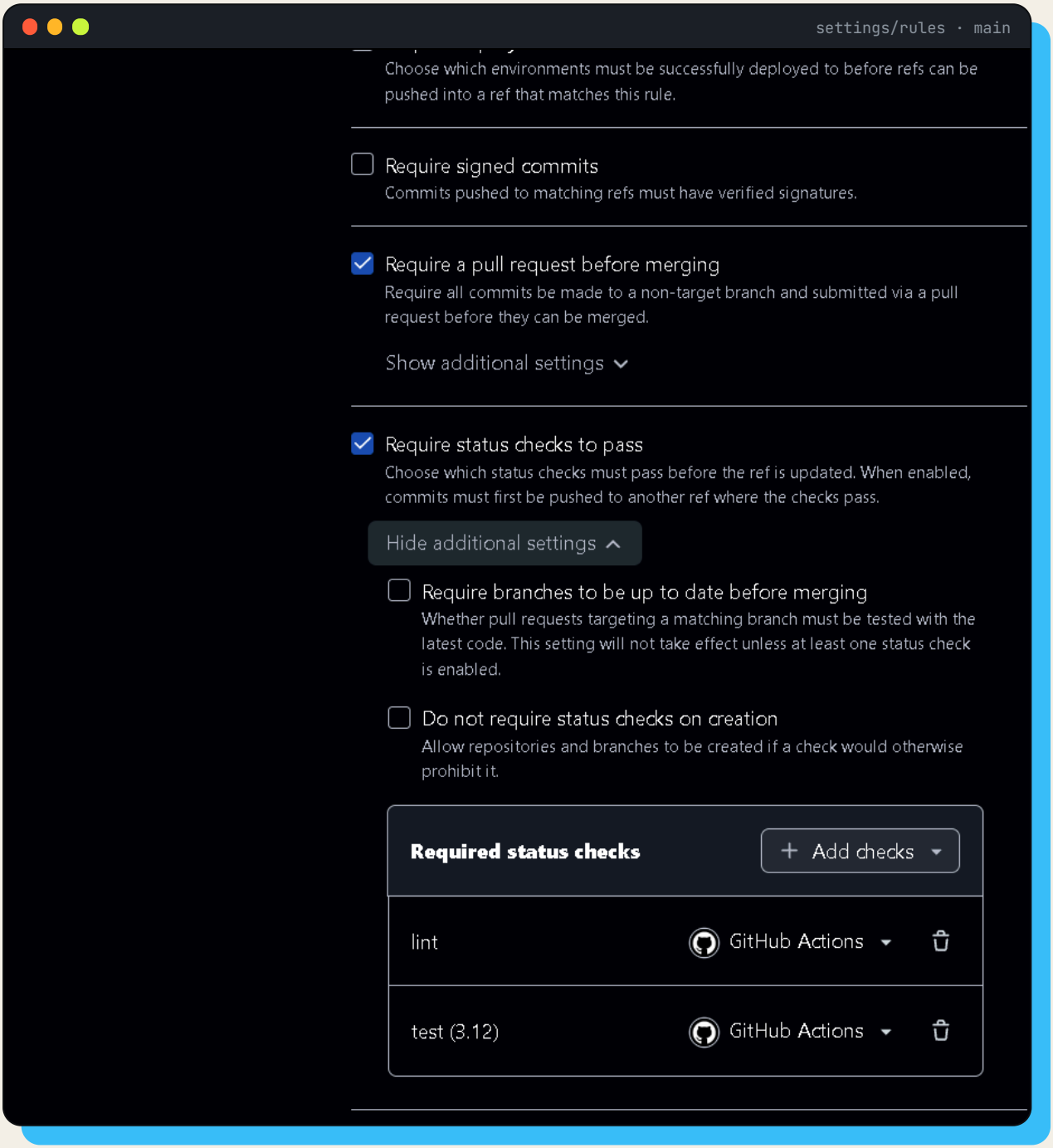




PROTECTED

TP 2-5 — `main` sous bonne garde

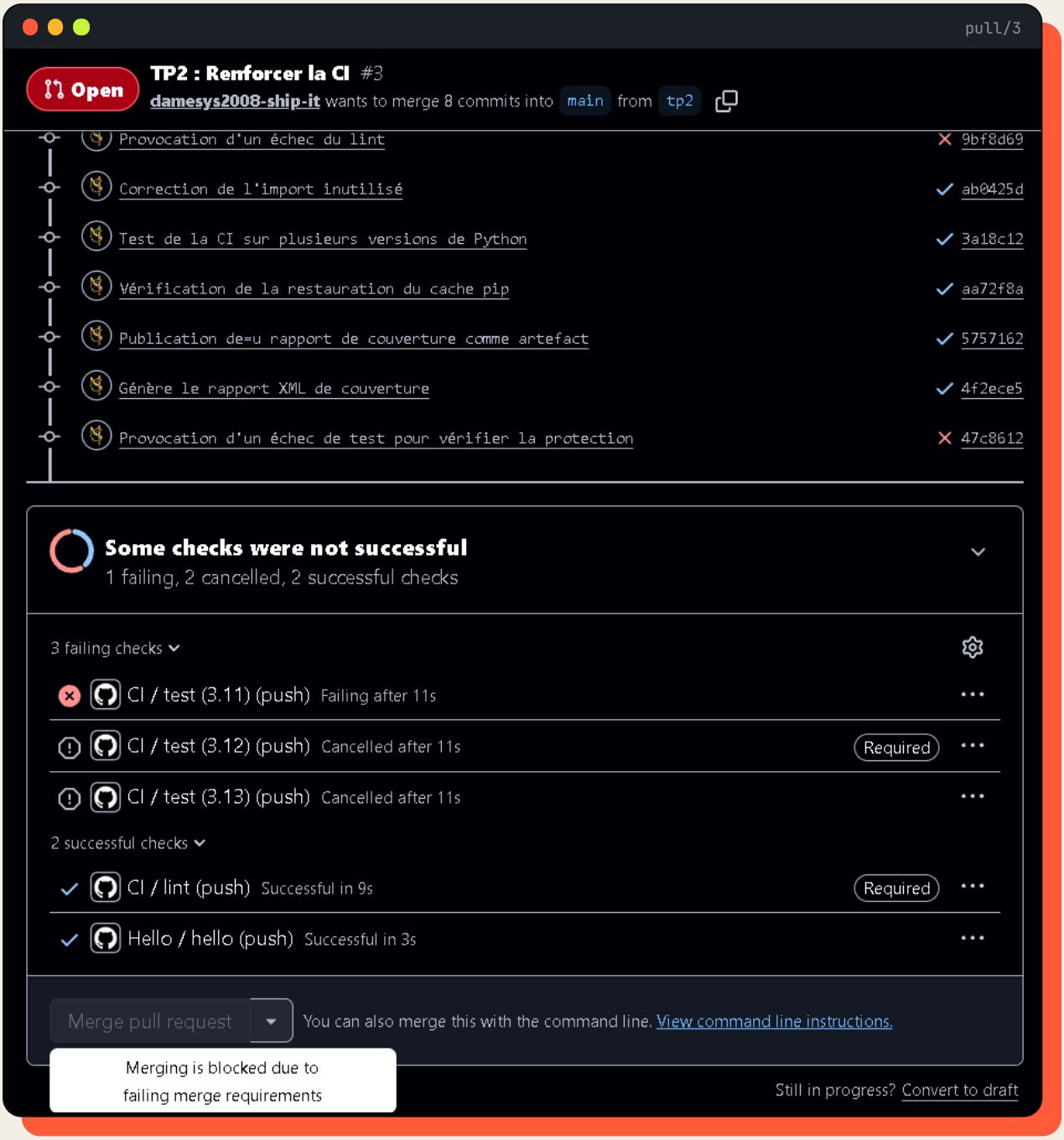
La règle de protection avec les vérifications obligatoires `lint` et `test (3.12)`.



MERGE BLOCKED

TP 2-6 Fusion refusée à l'entrée

Une pull request dont la fusion est bloquée par une vérification en échec.

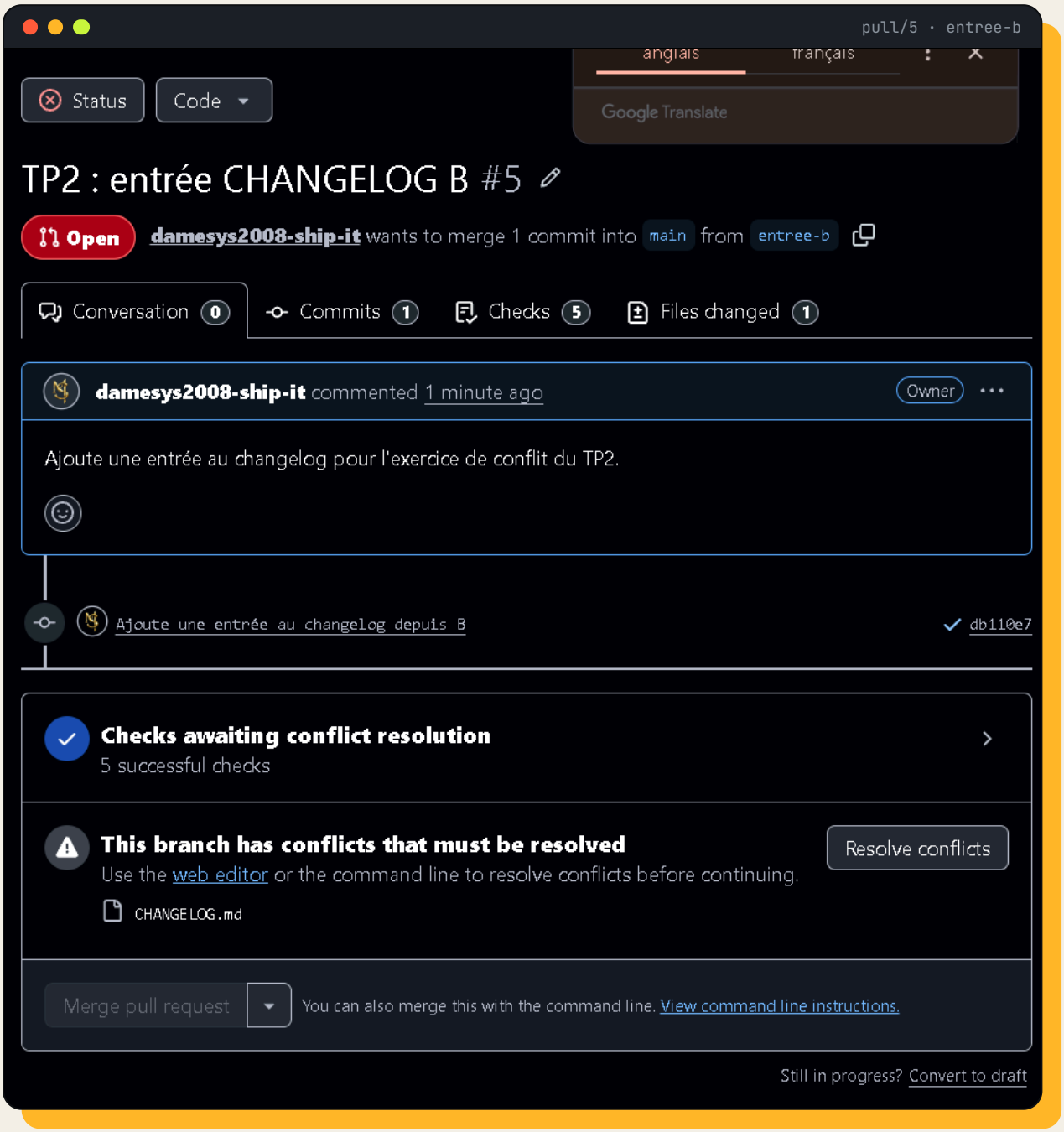


TP 2-7

CONFLICT

Deux lignes, une seule place

La PR de B avec le message de conflit sur `CHANGELOG.md`.

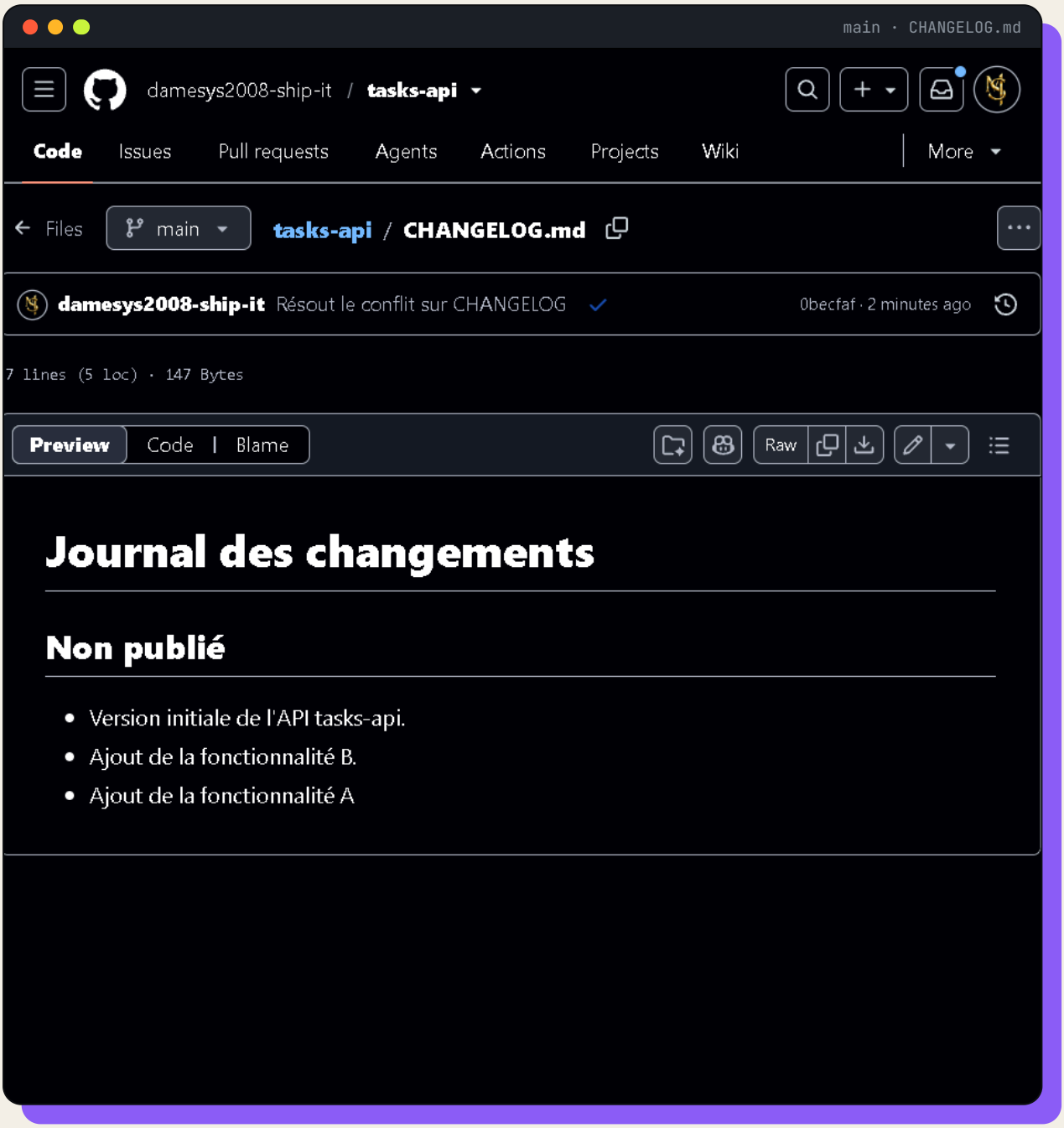


TP 2-8

MERGED

Tout le monde a sa ligne

Le fichier `CHANGELOG.md` final sur `main`, avec les deux lignes.

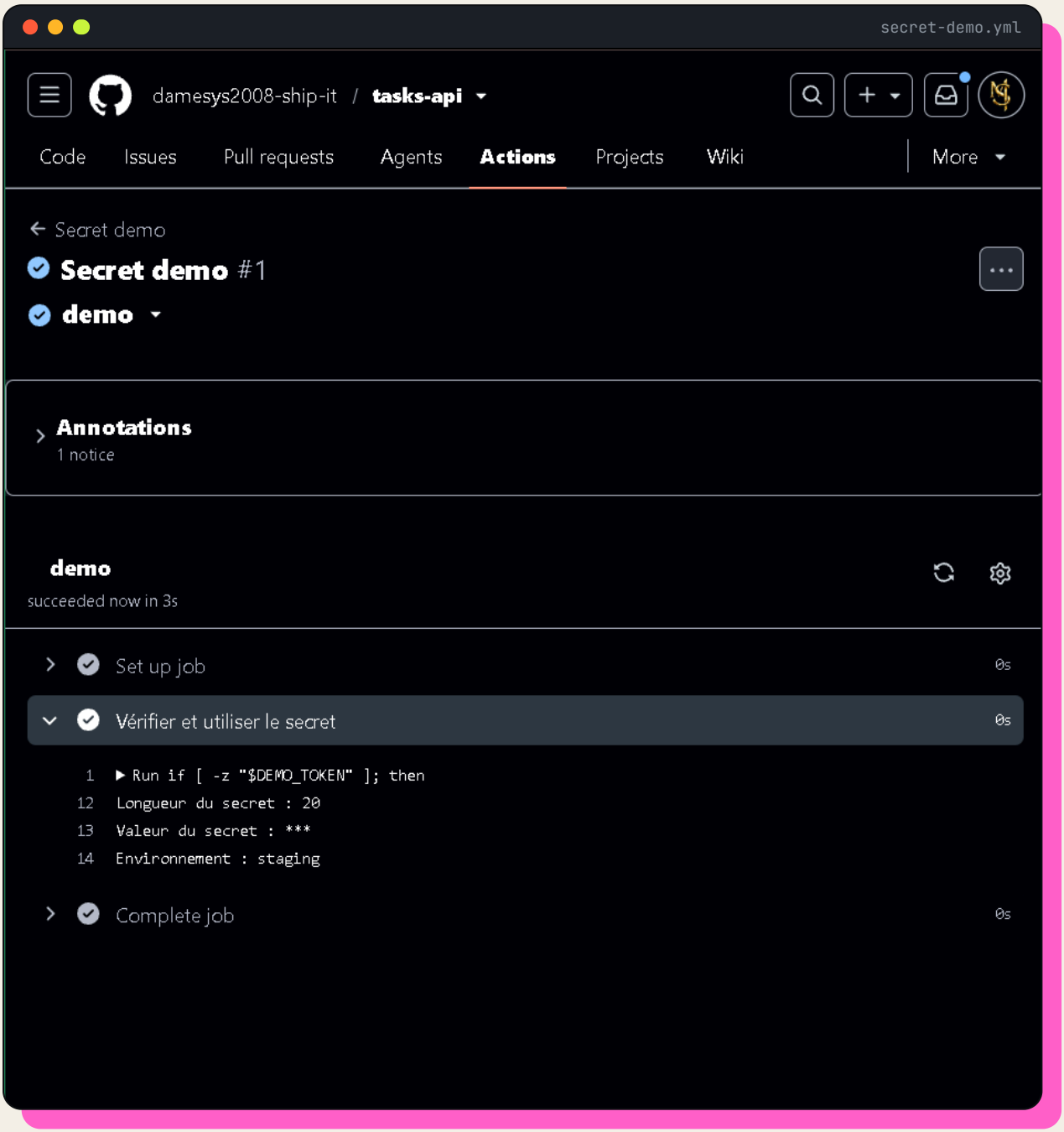


TP 2-9

✱ MASKED

Chut, c'est un secret

Les logs où la valeur du secret apparaît masquée par `***`.

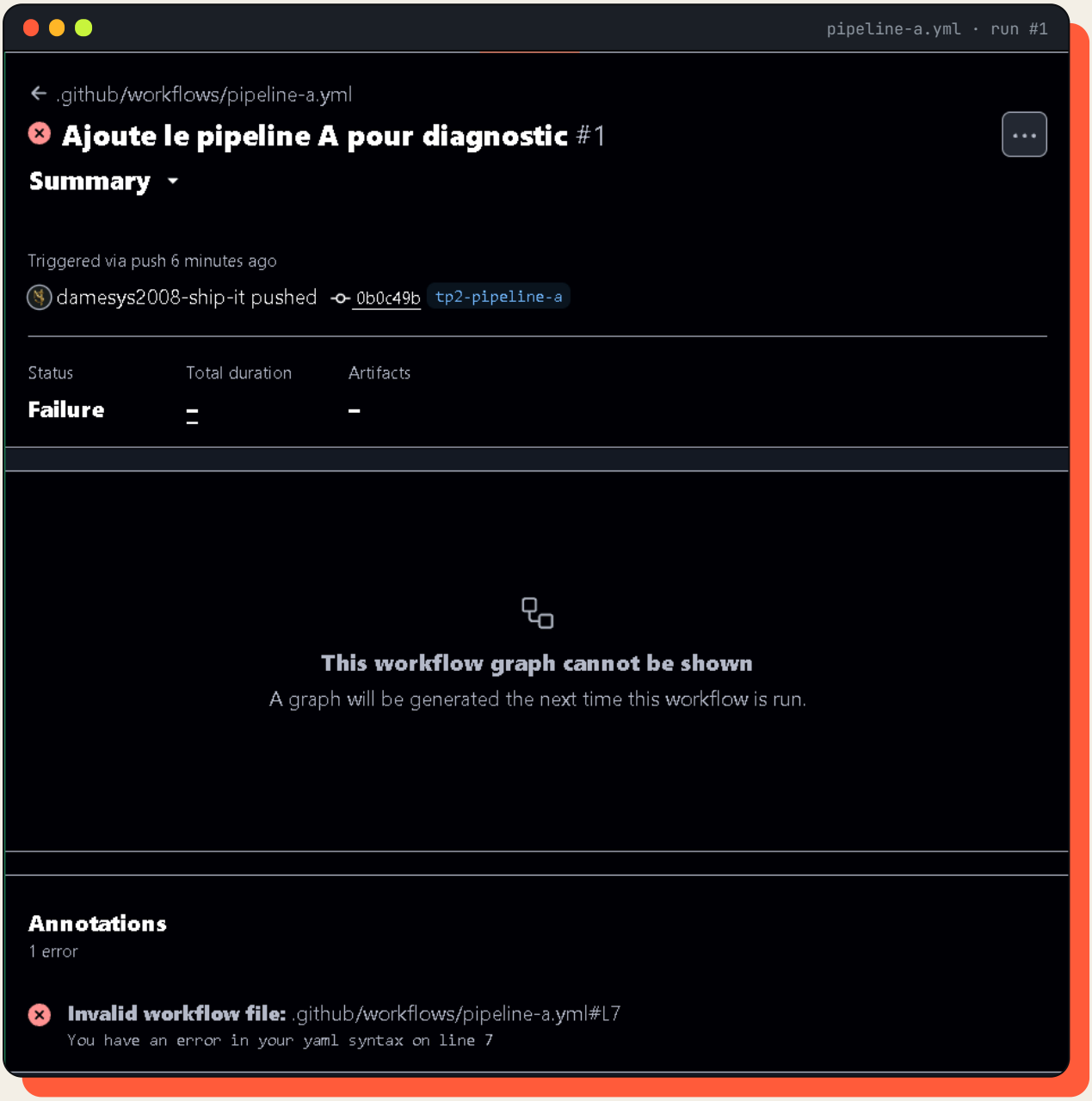


TP 2-10A

FAILED

Pipeline A : l'indentation qui fâche

Le run en échec du pipeline A : fichier de workflow invalide, erreur de syntaxe YAML ligne 7.

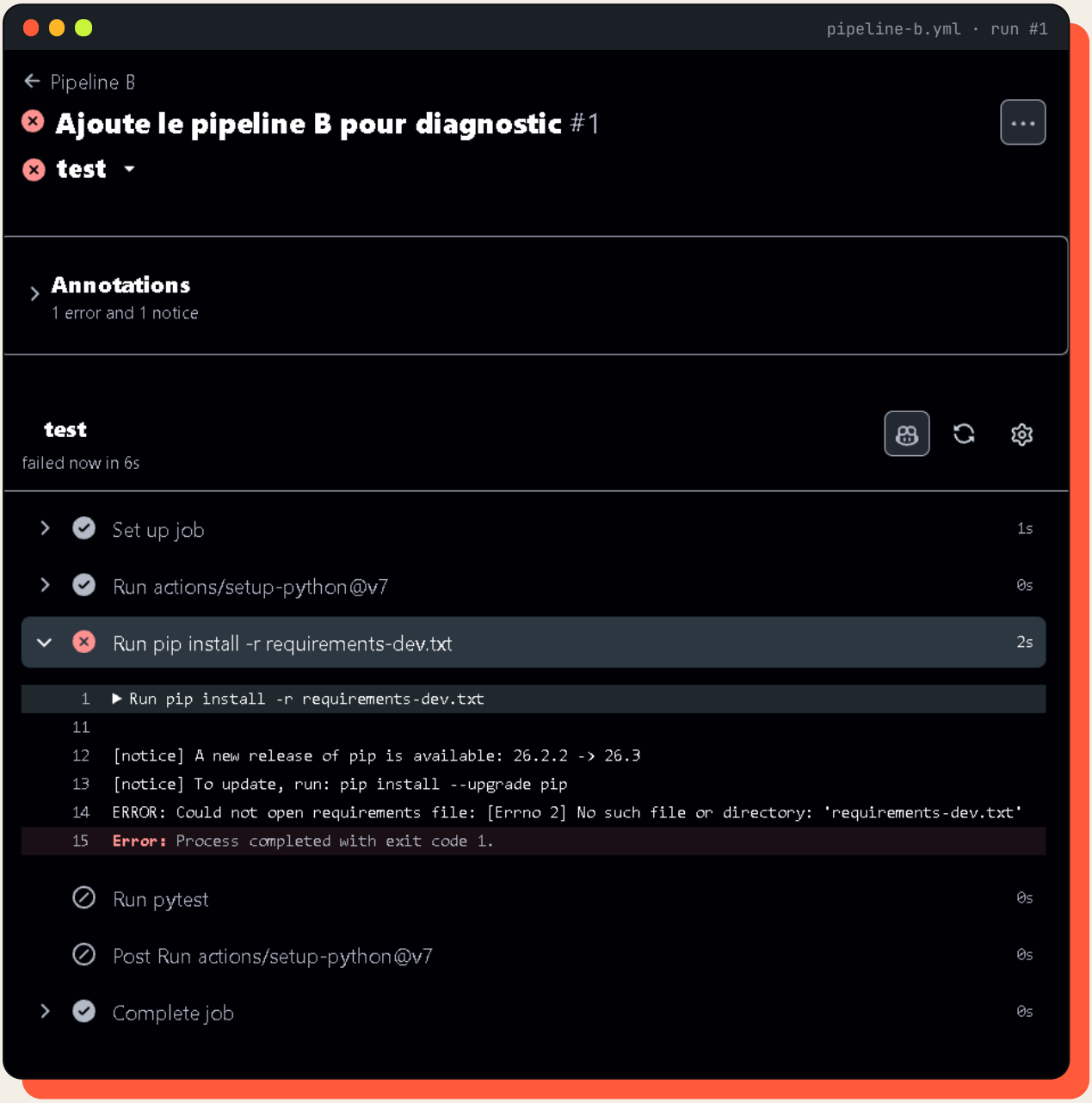


TP 2-10B

FAILED

Pipeline B : le checkout oublié

Le run en échec du pipeline B : `requirements-dev.txt` introuvable sur le runner.

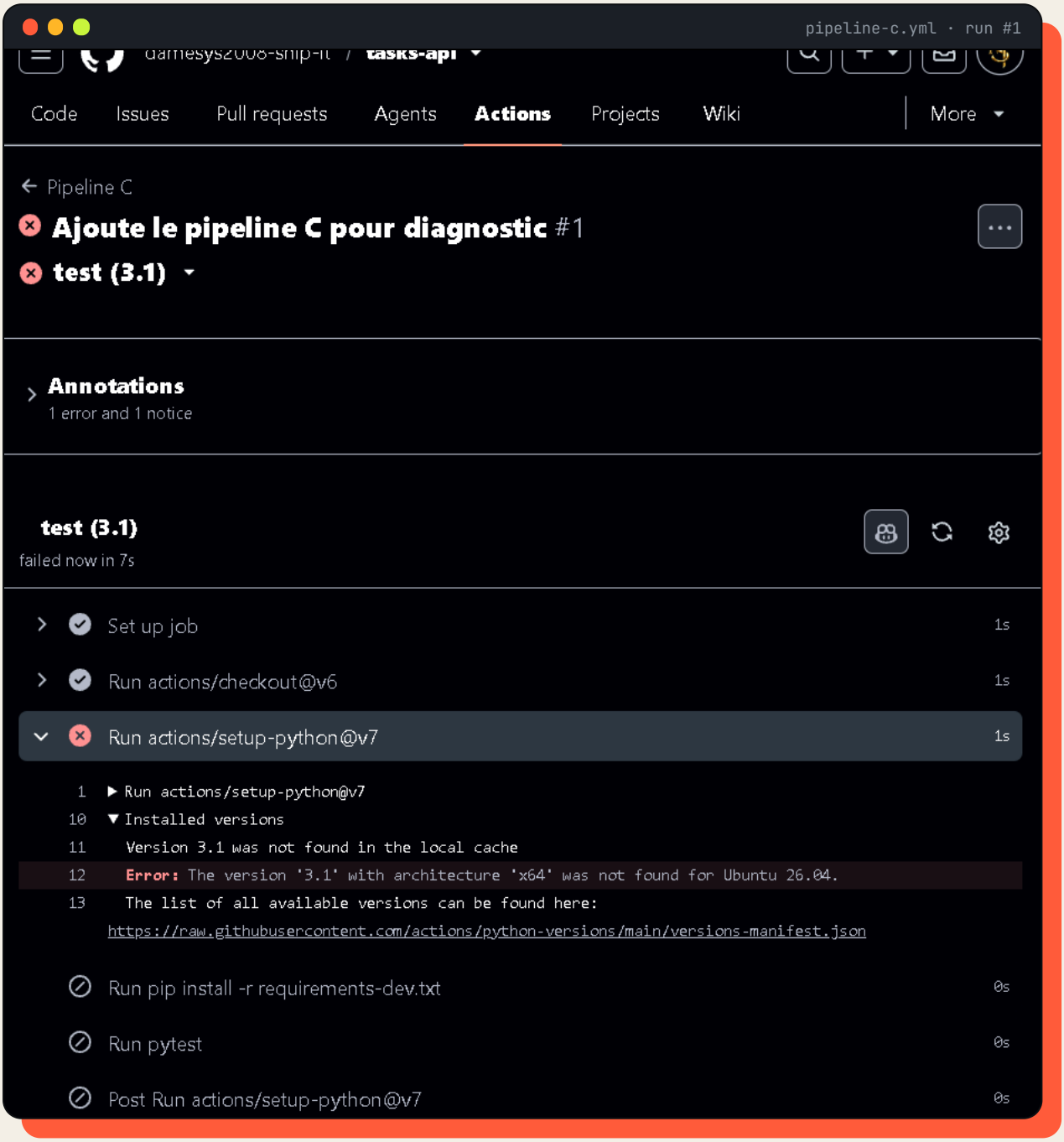


TP 2-10C

FAILED

Pipeline C : 3.10 = 3.1 ?!

Le run en échec du pipeline C : la version `'3.1'` introuvable pour setup-python.

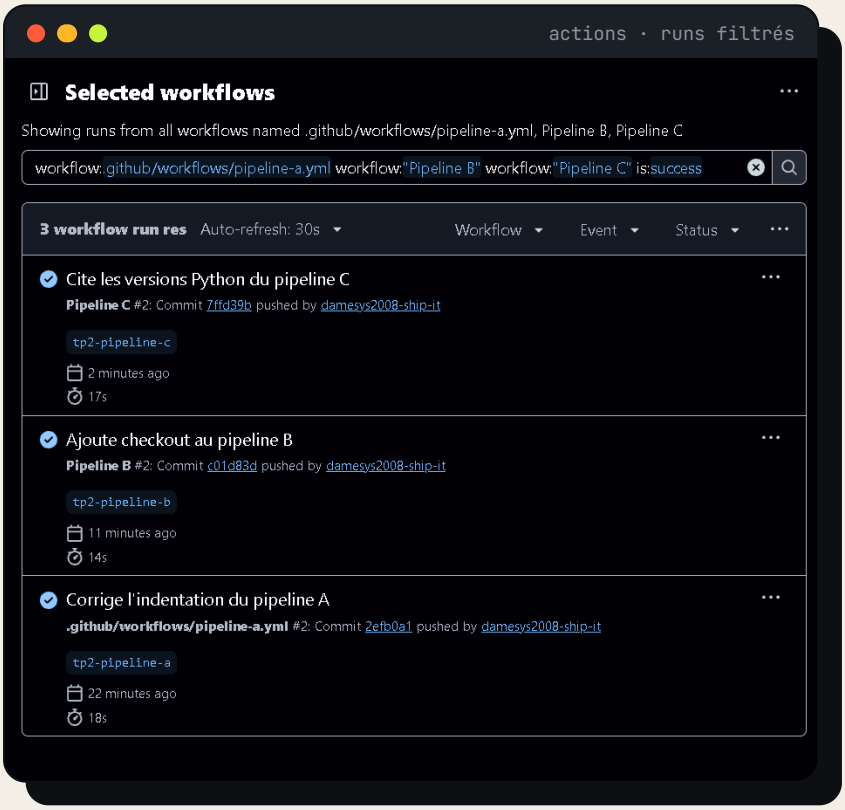


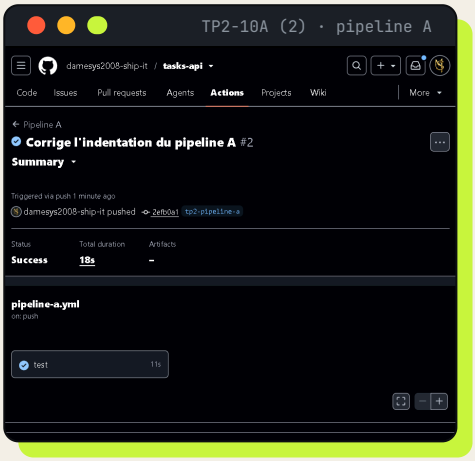
✓ ALL GREEN

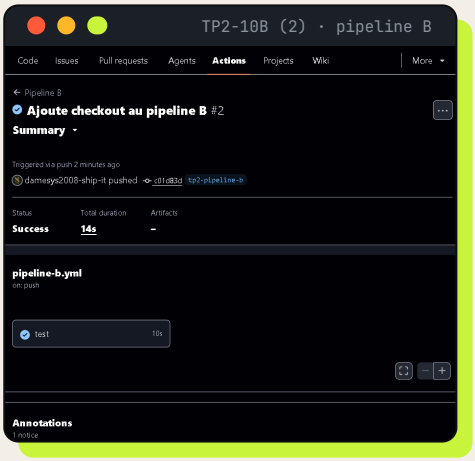
TP 2-11

Trois pipelines, trois feux verts

La liste des runs montrant les trois pipelines réparés au vert, avec à côté le run réparé de chacun (TP2-10A (2), TP2-10B (2), TP2-10C (2)).









$ cat tp2/reponses.md

Réponses
aux questions — TP2

PARTIE 1 — QUALITÉ AUTOMATIQUE

01

Quelle différence entre flake8 et Black ?

Flake8 analyse le code pour détecter des problèmes de style ou certaines erreurs, comme un import inutilisé. **Black** formate automatiquement le code selon des règles cohérentes. Dans la CI, `black --check` vérifie le formatage sans modifier les fichiers.

02

Pourquoi séparer lint et test en deux jobs ?

Cela distingue les vérifications de qualité des tests fonctionnels. Les jobs peuvent s'exécuter **en parallèle** et, si l'un échoue, ses résultats sont plus faciles à repérer sans confondre les causes.

PARTIE 2 — MATRICE, CACHE ET ARTEFACT

03

Pourquoi tester sur plusieurs versions de Python ?

Pour vérifier que l'application et ses dépendances fonctionnent sur **toutes les versions prises en charge**, et repérer une incompatibilité qui pourrait passer inaperçue avec une seule version.

04

Que gagne-t-on avec le cache ?

Le cache évite de retélécharger certains paquets déjà utilisés. Il peut ainsi **réduire le temps d'installation** et accélérer les runs suivants.

05

Un artefact, c'est quoi ?

C'est un **fichier produit par le workflow** et conservé avec le run pour pouvoir le consulter ou le télécharger ensuite. Ici, `coverage.xml` est le rapport de couverture.

PARTIE 3 — PROTECTION DE MAIN

06 Pourquoi interdire de pousser directement sur main ?

Pour faire passer les changements par une **pull request** et les **vérifications obligatoires** avant leur intégration. Cela réduit le risque d'introduire sur la branche principale un changement qui casse le projet.

PARTIE 4 — CONFLIT

07 Pourquoi Git ne peut-il pas résoudre ce conflit seul ?

Les deux branches ont modifié **la même zone du fichier** différemment. Git ne peut pas savoir si les deux changements doivent être conservés ni quel ordre est souhaité : il faut donc choisir et combiner les lignes manuellement.

08 Qu'est-ce qui a vérifié le code après la résolution ?

Les vérifications **GitHub Actions** lancées sur la pull request après le push de la résolution — notamment `lint` et les tests de la matrice — ont contrôlé le résultat avant la fusion.

PARTIE 5 — SECRETS ET VARIABLES

09 Pourquoi GitHub masque-t-il le secret ?

Pour éviter qu'une valeur enregistrée comme secret soit **révélée accidentellement dans les logs**. Ce masquage est une protection supplémentaire, pas une raison d'afficher un vrai secret.

10 Que se passe-t-il si on le commite dans le dépôt ?

La valeur peut être exposée dans le code et **rester accessible dans l'historique Git**, même si on la supprime ensuite. Il faut alors la révoquer ou la renouveler et la stocker dans les secrets GitHub, pas dans le dépôt.

11 Secret ou variable : comment choisir ?

Un **secret** sert aux données confidentielles, comme un jeton ou un mot de passe. Une **variable** convient à une configuration non sensible, comme le nom d'un environnement (`staging`).

PARTIE 6 — PIPELINES A, B ET C

PIPELINE **A**

✕ BUG & INDICE OBSERVÉ

L'indentation d'une étape YAML était décalée. GitHub a signalé une **erreur de syntaxe à la ligne 7** et le workflow ne pouvait pas s'exécuter correctement.

✓ COMMENT L'ÉVITER

Garder une indentation YAML cohérente et vérifier la syntaxe du workflow avant de le pousser.

PIPELINE **B**

✕ BUG & INDICE OBSERVÉ

L'étape `actions/checkout` manquait. Le runner ne trouvait donc pas **`requirements-dev.txt`** lors de l'installation.

✓ COMMENT L'ÉVITER

Récupérer le dépôt avec `actions/checkout` avant d'utiliser ses fichiers.

PIPELINE **C**

✕ BUG & INDICE OBSERVÉ

Les versions de la matrice n'étaient pas entre guillemets. YAML a interprété `3.10` comme le **nombre 3.1**, que setup-python ne pouvait pas installer.

✓ COMMENT L'ÉVITER

Écrire les versions comme chaînes, par exemple `"3.10"`, et vérifier leur valeur dans la matrice.

TP2 — 3/3 pipelines réparés · `exit 0`